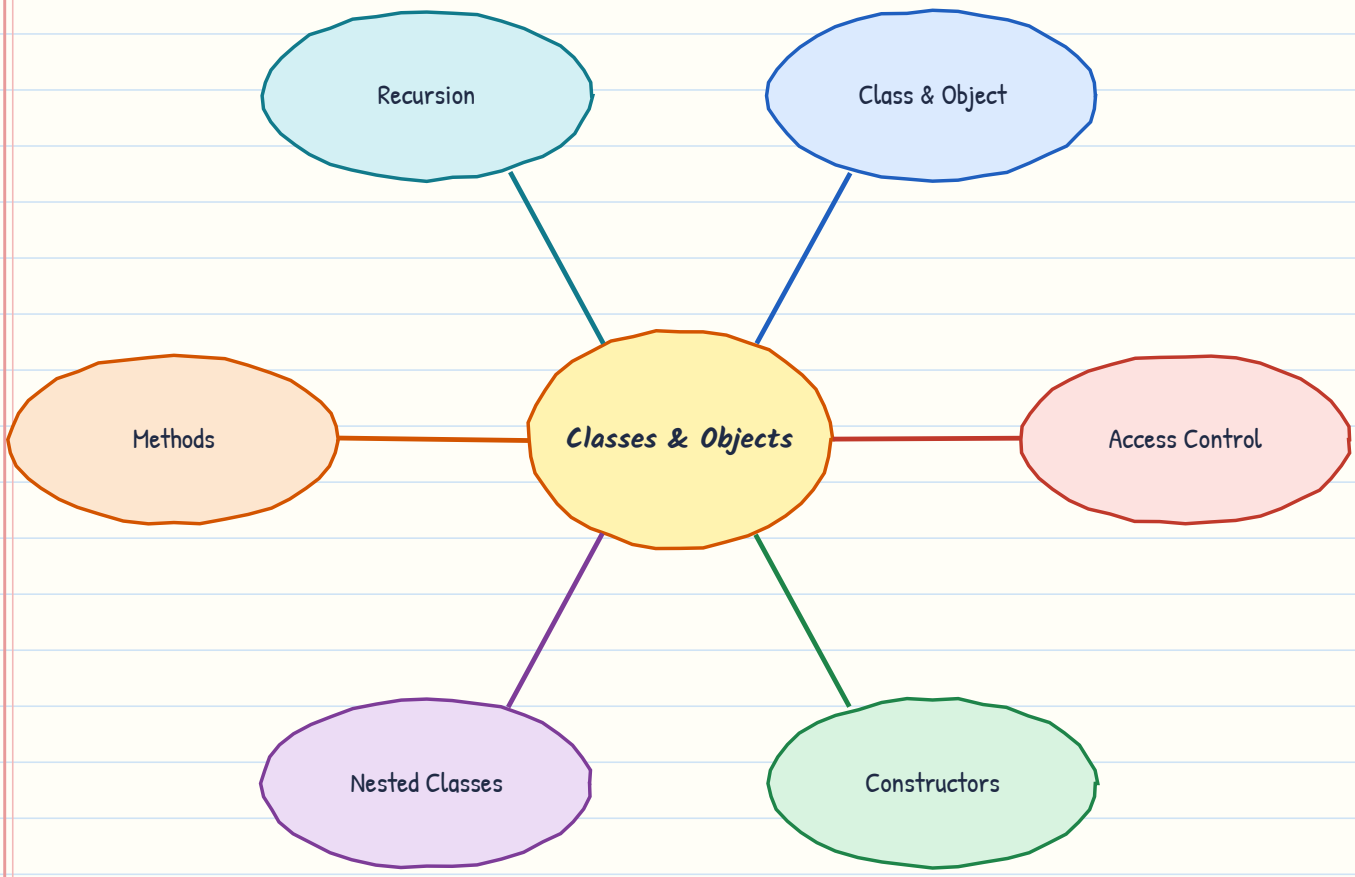
UNIT II

Classes and Objects, Methods

Declaration | Constructors | Access control | Overloading | Recursion



Mind map: everything you will learn in this unit

How to use these notes

Tip: read the diagrams first, then the text. Try writing each example yourself - code you type sticks in memory, code you only read does not.

I. Classes and Objects

A ***class*** is a user-defined blueprint that groups data (fields) and code (methods). An ***object*** is a real, memory-occupying instance of that class.

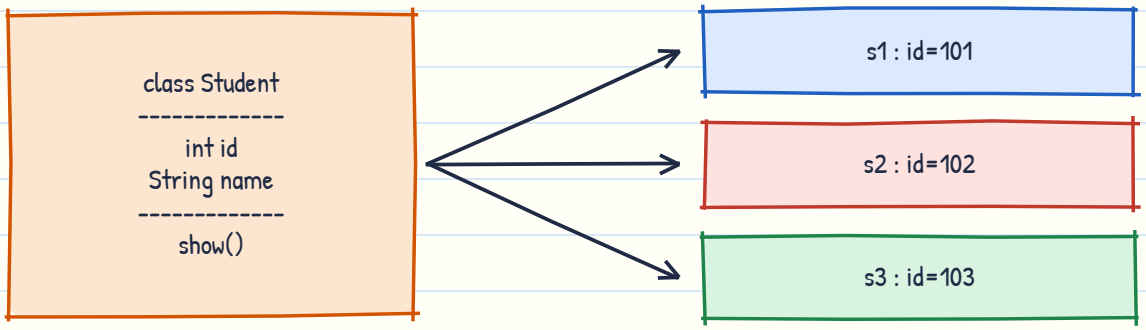


Fig 1: One class -> many objects (like one cake mould -> many cakes)

Class declaration & modifiers

```
[modifier] class ClassName {
    // fields (variables)
    // constructors
    // methods
}
```

Modifier	Meaning
public	class visible to every other class
(default)	visible only inside same package
abstract	cannot create objects; meant to be extended
final	cannot be inherited (no subclass)

Class members & creating objects

Student.java

```java
class Student {
    int id;  String name;            // instance variables
    void show() {                    // instance method
        System.out.println(id + " " + name);
    }
    public static void main(String[] args) {
        Student s1 = new Student();  // create object
        s1.id = 101;  s1.name = "Asha";
        s1.show();                   // call method
    }
}
```

```
> OUTPUT
101 Asha
```

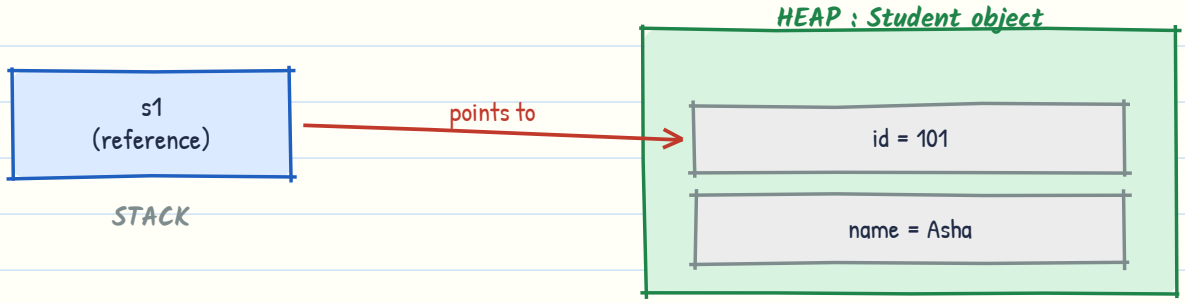


Fig 2: Student s1 = new Student(); - reference vs object in memory

Key idea

new allocates memory in the HEAP and returns its address. The variable s1 only stores that address (a reference), not the object itself.

Assigning one object to another

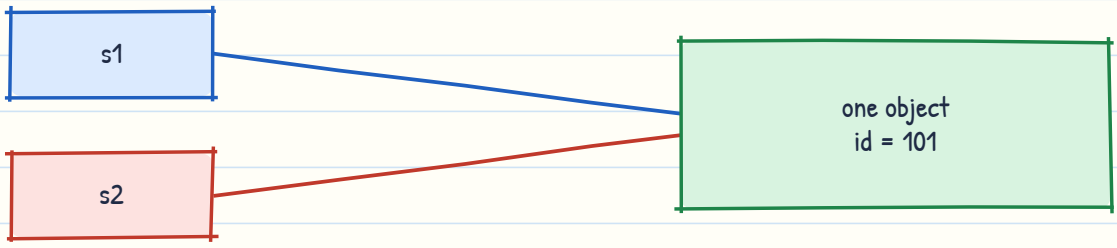


Fig 3: s2 = s1 copies the REFERENCE, not the object

```java
Student s1 = new Student();  s1.id = 1;
Student s2 = s1;             // both point to same object
s2.id = 99;
System.out.println(s1.id);   // 99  (changed through s2!)
```

```
> OUTPUT
99
```

2. Access Control

Access modifiers decide *who can see* a member. Remember the order: private < default < protected < public.

Modifier	Same class	Same package	Subclass (other pkg)	Anywhere
private	YES	no	no	no
default	YES	YES	no	no
protected	YES	YES	YES	no
public	YES	YES	YES	YES

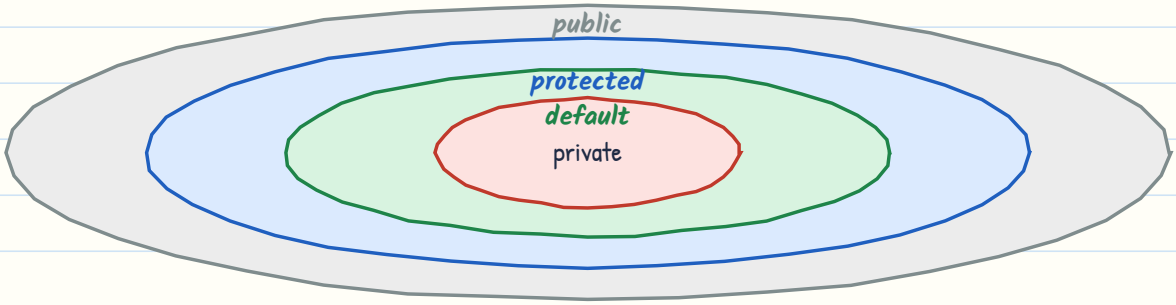


Fig 4: Visibility circles

Accessing private members (getters & setters)

Make fields `private` and give controlled access through public methods - this is *encapsulation* in action.

Account.java

```
class Account {
    private double balance;                 // hidden
    public double getBalance() { return balance; }        // getter
    public void setBalance(double b) {                    // setter
        if (b >= 0) balance = b;            // validation!
    }
}
class Test {
    public static void main(String[] a) {
        Account ac = new Account();
        ac.setBalance(5000);
        // ac.balance = 10;   ERROR: balance has private access
        System.out.println(ac.getBalance());
    }
}
```

```
> OUTPUT
5000.0
```

3. Constructors

A ***constructor*** is a special method that runs automatically when an object is created, used to initialise fields.

- Same ***name as the class***; has ***no return type*** (not even void).
- Called automatically by **new**.
- If you write none, Java gives a hidden ***default constructor***.
- Cannot be **static**, **final** or **abstract**.

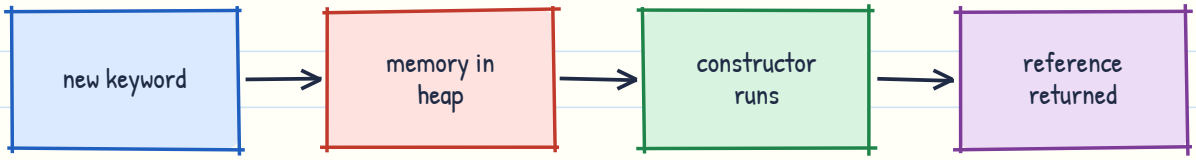


Fig 5: What happens at new Student("Ravi", 21)

```java
class Student {
    String name; int age;
    Student() { name = "Unknown"; age = 0; }          // default (no-arg)
    Student(String n, int a) { name = n; age = a; }   // parameterized
    Student(Student s) { name = s.name; age = s.age; } // copy
}
```

Overloaded constructors

Several constructors with the ***same name but different parameter lists*** – Java picks the one matching the arguments.

```java
Student a = new Student();                 // calls 1st
Student b = new Student("Ravi", 21);       // calls 2nd
Student c = new Student(b);                // calls 3rd
```

Keyword this

`this` is a reference to the ***current object***. Uses: (1) separate field from parameter with same name, (2) call another constructor `this(...)`, (3) pass current object.

```java
class Box {
    int w, h;
    Box(int w, int h) { this.w = w; this.h = h; }   // this.w = field
    Box(int s)        { this(s, s); }               // calls Box(int,int)
}
```

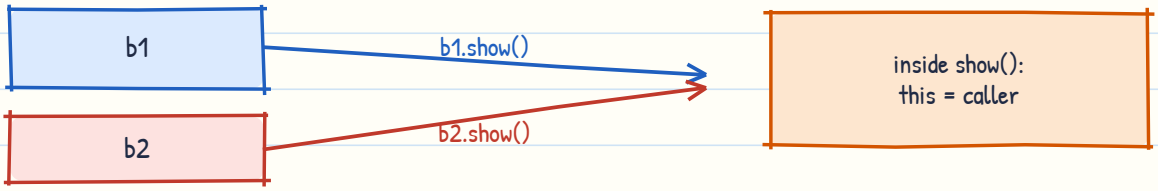


Fig 6: this points to whichever object is calling

4. Nested Classes & final

A class declared ***inside another class*** is a nested class. It helps group classes used only in one place.

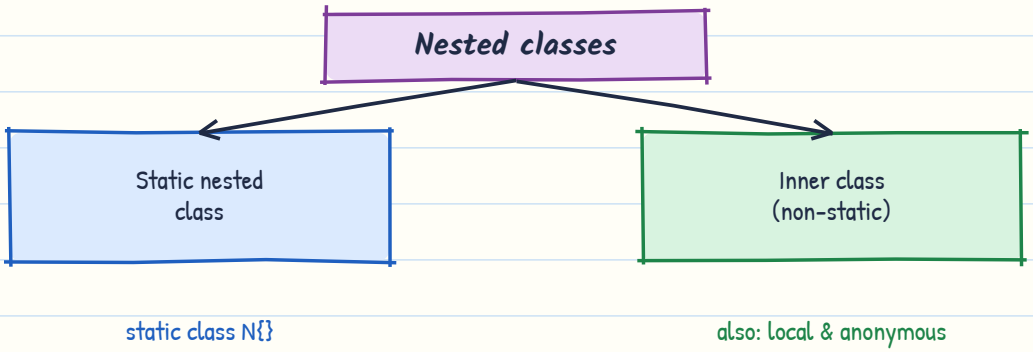


Fig 7: Types of nested classes

```
class Outer {
    int x = 10;
    class Inner {                          // inner class
        void show() { System.out.println("x = " + x); }  // sees Outer's members
    }
    static class SNested { void hi() { System.out.println("static nested"); } }
    public static void main(String[] a) {
        Outer o = new Outer();
        Outer.Inner in = o.new Inner();          // needs outer object
        in.show();
        new Outer.SNested().hi();                // no outer object needed
    }
}
```

```
> OUTPUT
x = 10
static nested
```

Final class and final method

Use	Effect	Example
final variable	value cannot change	final int MAX = 5;
final method	cannot be overridden	final void show(){}
final class	cannot be extended	final class Util{} (e.g. String)

Passing arguments: by value / by reference

Java is ***always pass-by-value***. For primitives the *value* is copied; for objects the *reference* (address) is copied - so the method can change the object's content but cannot re-point the caller's variable.

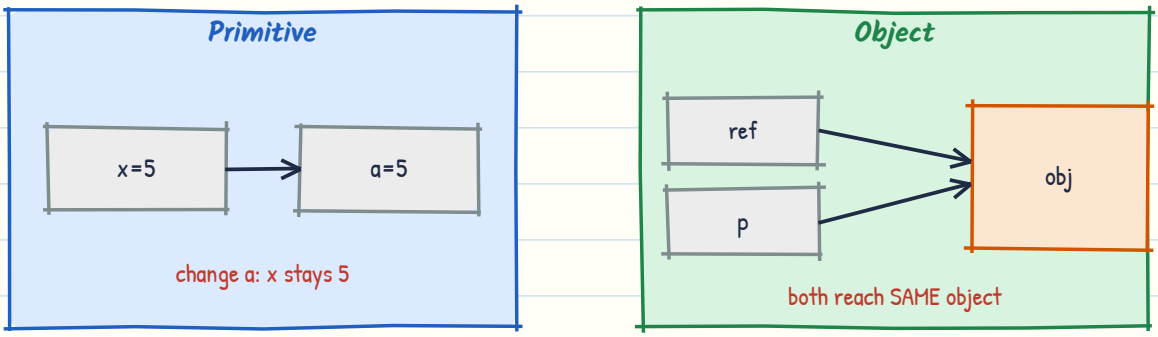


Fig 8: primitive copy vs object reference copy

```
static void change(int a)     { a = 50; }          // original unchanged
static void change(Box b)     { b.w = 50; }        // original CHANGED
int x = 5;  change(x);        System.out.println(x);      // 5
Box bx = new Box(1,1);  change(bx);  System.out.println(bx.w); // 50
```

```
> OUTPUT
5
50
```

5. Methods

A *method* is a named block of code that performs one task and can be reused. It reduces repetition and makes programs modular.

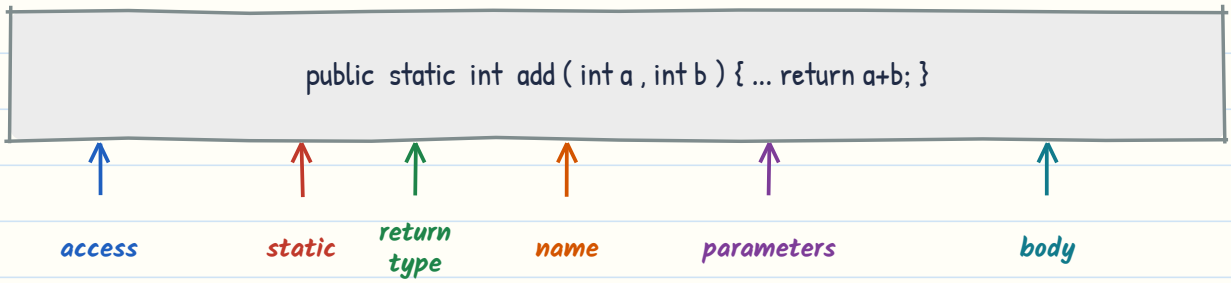


Fig 9: Anatomy of a method

```java
static int add(int a, int b) {     // defining
    return a + b;
}
public static void main(String[] args) {
    int r = add(4, 6);               // calling
    System.out.println(r);
}
```

```
> OUTPUT
10
```

- *Parameters* = variables in the definition; *arguments* = real values passed in the call.
- `void` methods return nothing; others must `return` a matching value.

Method overloading (compile-time polymorphism)

Same method name in one class with *different parameter lists* (number, type or order). Return type alone is NOT enough.

```java
int    add(int a, int b)          { return a + b; }
double add(double a, double b)    { return a + b; }
int    add(int a, int b, int c)   { return a + b + c; }
// add(2,3) -> 1st   add(2.5,3.5) -> 2nd   add(1,2,3) -> 3rd
```

Objects as parameters & returning objects

```java
class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }
    Point add(Point p) {                       // object as parameter
        return new Point(x + p.x, y + p.y);    // object returned
    }
}
Point r = new Point(1,2).add(new Point(3,4));  // r = (4,6)
```

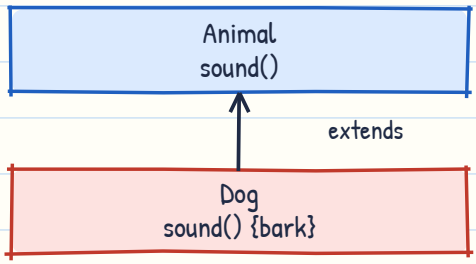
Access control for methods & nesting

Methods use the same modifiers as fields. *Nesting of methods:* Java does not allow a method *inside* a method, but one method may *call* another.

```java
static int square(int n) { return n * n; }
static int sumSquares(int a, int b) { return square(a) + square(b); }  // nested call
// sumSquares(3,4) = 9 + 16 = 25
```

Method overriding

A subclass redefines a method of its parent with the **same name, parameters and return type** (runtime polymorphism). Use `@Override`.



Animal a = new Dog();

a.sound() -> "Woof"

child version runs!

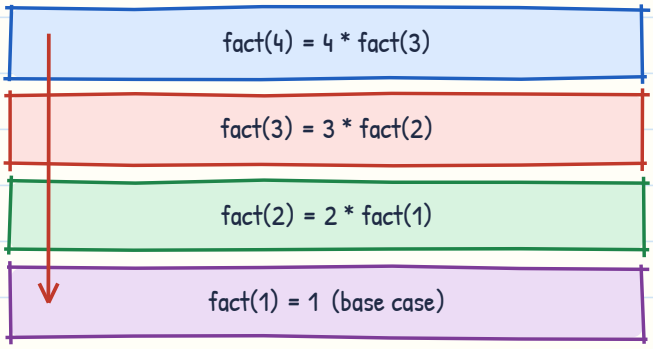
Fig 10: Overriding

	Overloading	Overriding
Where	same class	parent + child
Signature	must differ	must be same
Binding	compile time	run time
Purpose	many forms of input	change inherited behaviour

Recursive methods

A method that **calls itself**. It needs (1) a **base case** to stop and (2) a **recursive case** that moves towards it.

```
static int fact(int n) {
    if (n <= 1) return 1;          // base case
    return n * fact(n - 1);        // recursive case
}
// fact(4) = 24
```



returns 24

returns 6

returns 2

returns 1

calls go down,

answers return up

Fig 11: Call stack of fact(4)

> **Warning**
> No base case = infinite recursion = StackOverflowError. Always ask: "where does it stop?"

Static and final attributes - recap

Keyword	On variable	On method
static	one shared copy per class	call without object; no this
final	constant value	cannot be overridden

Quick Revision & Exam Questions

Topic	Memory line
Object	new -> heap; variable holds reference
s2 = s1	copies reference, one object only
Constructor	class name, no return type, auto-called
this	current object; this(...) calls another constructor
Pass by	Java is always pass-by-value (refs copied)
Overload / Override	compile-time / runtime polymorphism
Recursion	base case + recursive case

Likely exam questions

- Define class and object; write a program creating objects of a class.
- Explain access modifiers with a table and example.
- What is a constructor? Explain types and overloaded constructors.
- Explain keyword this with examples.
- Differentiate method overloading and overriding.
- Explain pass-by-value and pass-by-reference in Java.
- Write a recursive program for factorial / Fibonacci.
- Write a note on nested and inner classes.